Especificación de Arquitectura y Modelado UML (Figuras 1 a 7)

Proyecto: VERTEX — Plataforma Educativa de Ciberseguridad basada en Teoría de Grafos

Figura 1. Diagrama de Casos de Uso General del Sistema

La **Figura 1** presenta el modelo formal de Casos de Uso General del sistema **VERTEX** bajo el estándar UML 2.5. El diagrama ilustra los límites perimetrales del software, la interacción con el actor principal (Operador) y los servicios secundarios (el agente autónomo de Inteligencia Artificial que actúa como adversario reactivo y el subsistema de Persistencia en disco). Los casos de uso centrales descomponen la administración de perfiles, los módulos formativos de la academia, los tres modos operacionales (Heist, Hacker y Ciberdefensa) y los mecanismos de extensión funcional como la declaración de cuarentena y la revelación táctica de rutas.

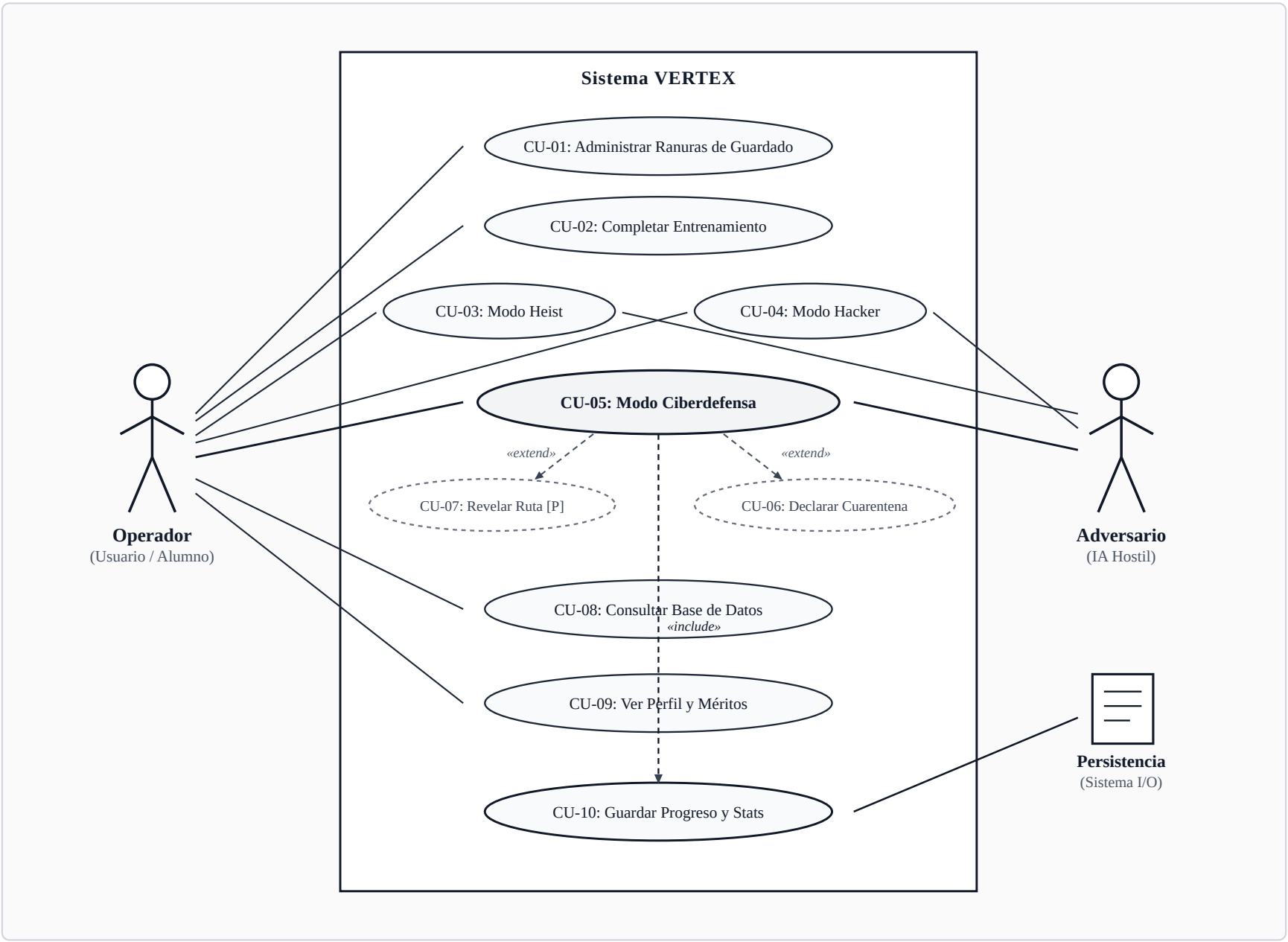


Figura 1. *Diagrama de Casos de Uso General del Sistema VERTEX según especificación UML 2.5.*

Figura 2. Diagrama de Secuencia del Ciclo Reactivo de Ciberdefensa

La **Figura 2** presenta el **diagrama de secuencia** del ciclo reactivo completo del modo Ciberdefensa (Blue Team). El flujo temporal modela la recepción del evento de bloqueo comandado por el analista en la interfaz gráfica (`InputHandler`), la delegación hacia el mediador de lógica (`GameLogic`), el sellado transaccional de aristas con costo infinito en la capa de grafo (`NetworkRuntime`), la resolución automática del turno en `DefenderBrain`, el cálculo y avance del rastreo forense EDR, la respuesta estocástica del atacante (`move_enemy()`) mediante movimiento lateral o exploit `Bypass`, y la actualización del estado visual en `GameRenderer`.

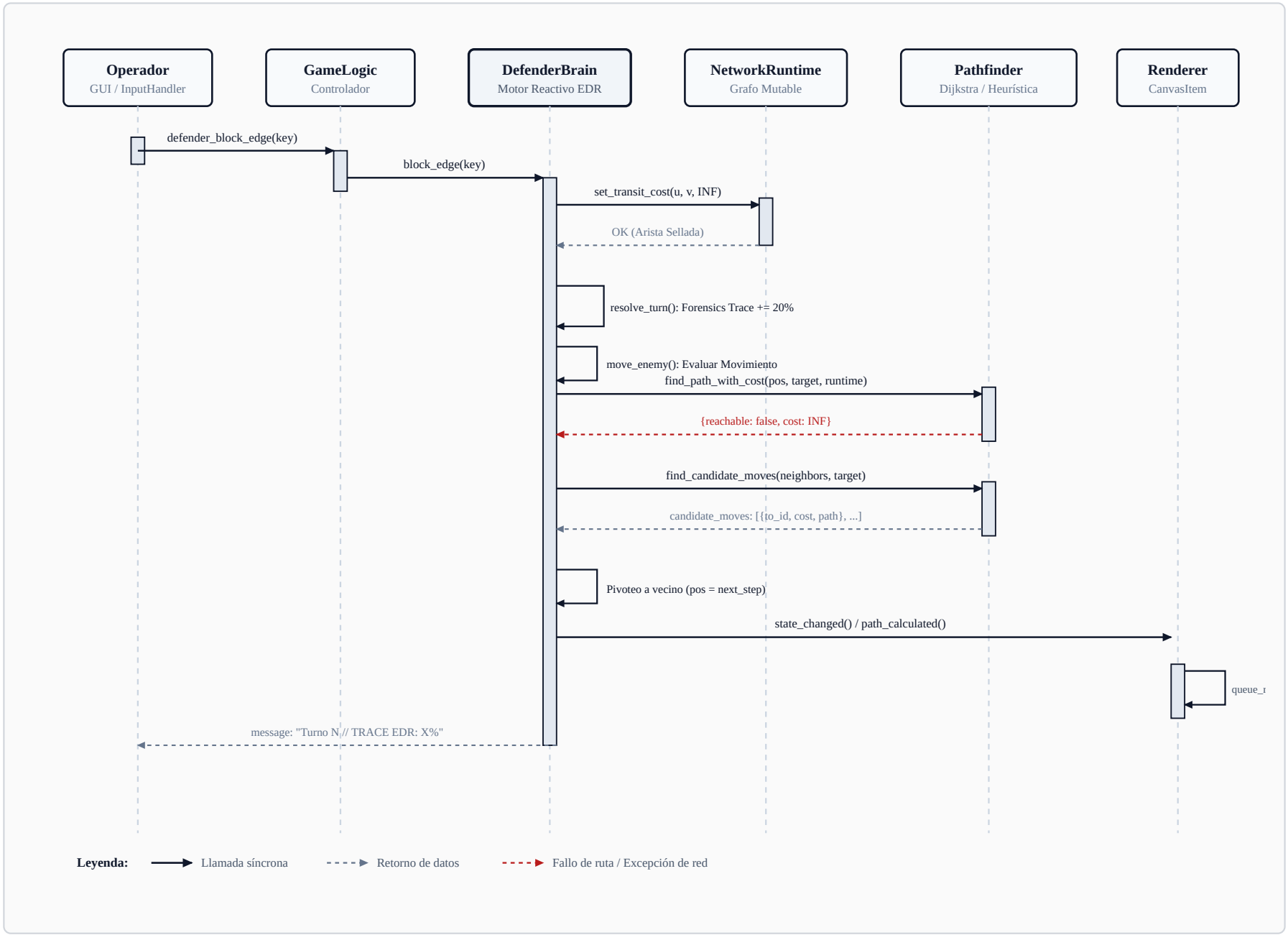


Figura 2. *Diagrama de secuencia del ciclo de respuesta reactiva 1:1 en modo Ciberdefensa.*

Figura 3. Diagrama de Actividades de Ciberdefensa (Procesos)

La **Figura 3** ilustra el **diagrama de actividades (flujo de proceso)** del ciclo de defensa perimetral según el estándar UML. El proceso modela el nodo inicial de inicio de asedio, la bifurcación condicional de acciones del operador (bloqueo de arista, colocación de firewall permanente o aislamiento de host en cuarentena), la resolución del turno de la Inteligencia Artificial mediante búsqueda de camino, la evaluación de contingencia por exploit `Bypass`, la ramificación de decisión por estado de rastreo forense (Trace EDR $\geq 100\%$) y los nodos finales de actividad correspondientes a la victoria por mitigación o la derrota por brecha en el Núcleo.

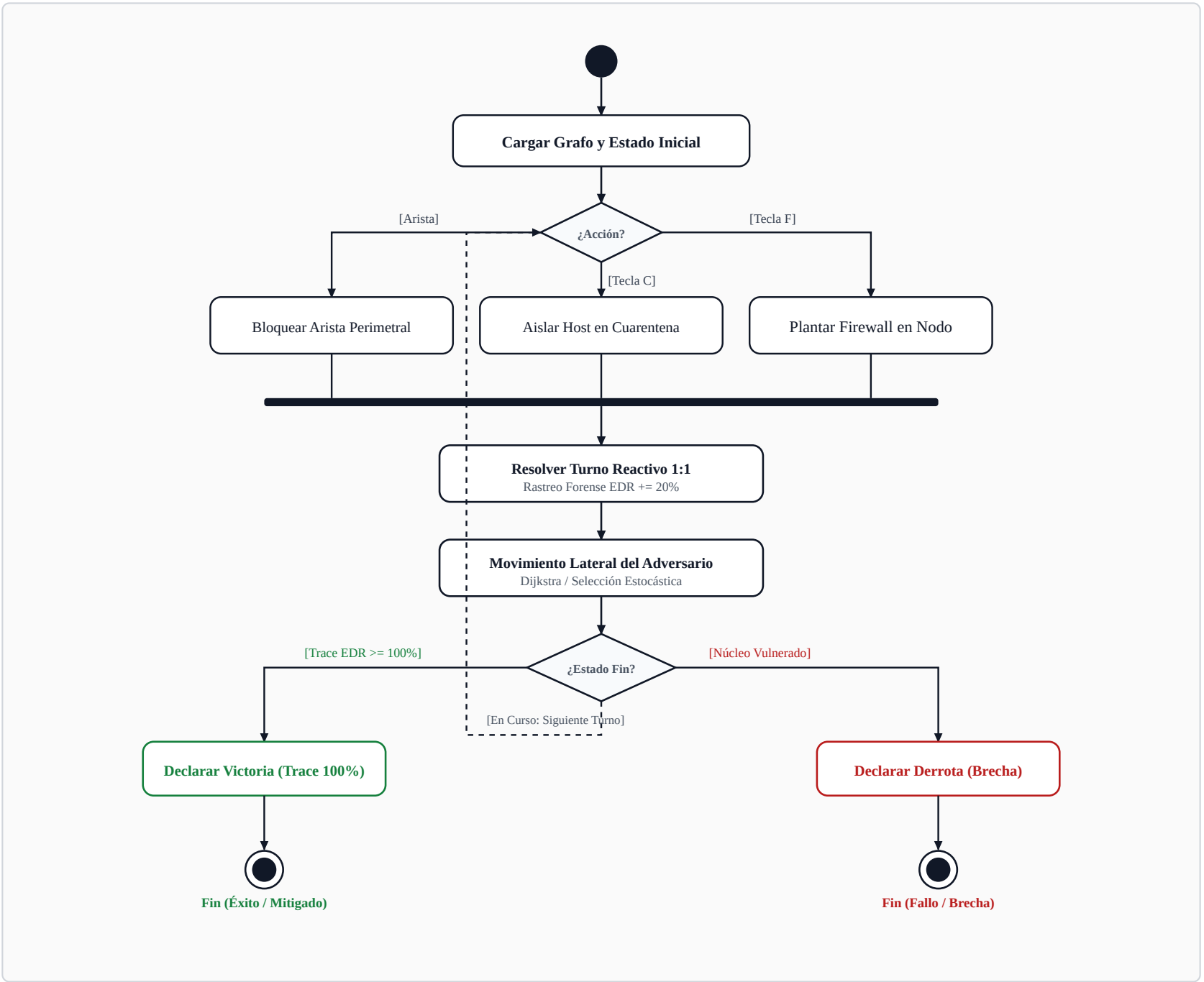

Cargar Grafo y Estado Inicial
¿Acción?
[Arista]
[Tecla F]
[Tecla C]
Bloquear Arista Perimetral
Aislar Host en Cuarentena
Plantar Firewall en Nodo
Resolver Turno Reactivo 1:1
Rastreo Forense EDR += 20%
Movimiento Lateral del Adversario
Dijkstra / Selección Estocástica
¿Estado Fin?
[Trace EDR >= 100%]
[Núcleo Vulnerado]
[En Curso: Siguiente Turno]
Declarar Victoria (Trace 100%)
Declarar Derrota (Brecha)
Fin (Éxito / Mitigado)
Fin (Fallo / Brecha)

Figura 3. *Diagrama de actividades del ciclo de resolución de Ciberdefensa según estándar UML.*

Figura 4. Diagrama de Actividades de Infiltración Ofensiva (Modo Hacker)

La **Figura 4** presenta el **diagrama de actividades** del proceso de infiltración ofensiva del modo Hacker (Red Team). Describe la secuencia de decisiones que gobierna el reconocimiento pasivo con `Nmap`, el consumo de exploits (`Bypass`, `Escalate`, `Persist`, `Decoy`), el cálculo y acumulación del nivel de ruido en la red perimetral, la evaluación de umbrales de detección ante la IA centinela (despacho de perseguidores), y la finalización del escenario con persistencia de estrellas de maestría por sigilo.

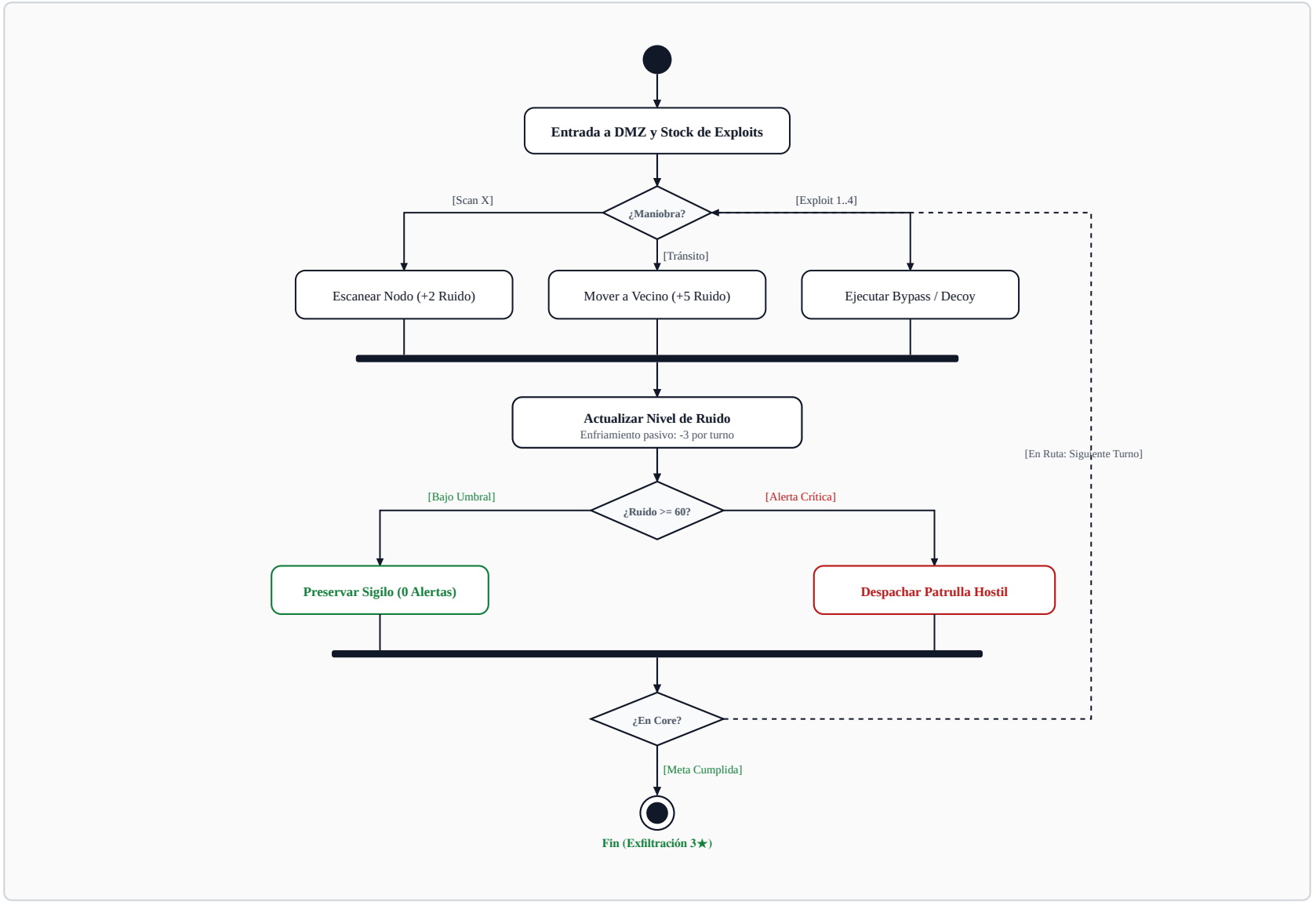


Figura 4. *Diagrama de actividades del ciclo de infiltración y balance de ruido en modo Hacker.*

Figura 5. Diagrama de Clases de Arquitectura (UML 2.5)

La **Figura 5** presenta el **diagrama de clases** del núcleo arquitectónico de **VERTEX** bajo el estándar UML 2.5. Modela la separación de responsabilidades entre los recursos inmutables del grafo (`NetworkGraphResource`, `NetworkNodeResource`, `NetworkEdgeResource`), la capa de mutación en memoria de tiempo de ejecución (`NetworkRuntime`), los algoritmos deterministas independientes (`DefensivePathfinder` y `StrategicAnalyzer`), y los controladores de dominio y presentación (`DefenderBrain`, `GameLogic` y `GameRenderer`). Se representan las relaciones de composición, agregación y dependencia unidireccional que garantizan desacoplamiento modular en el motor.

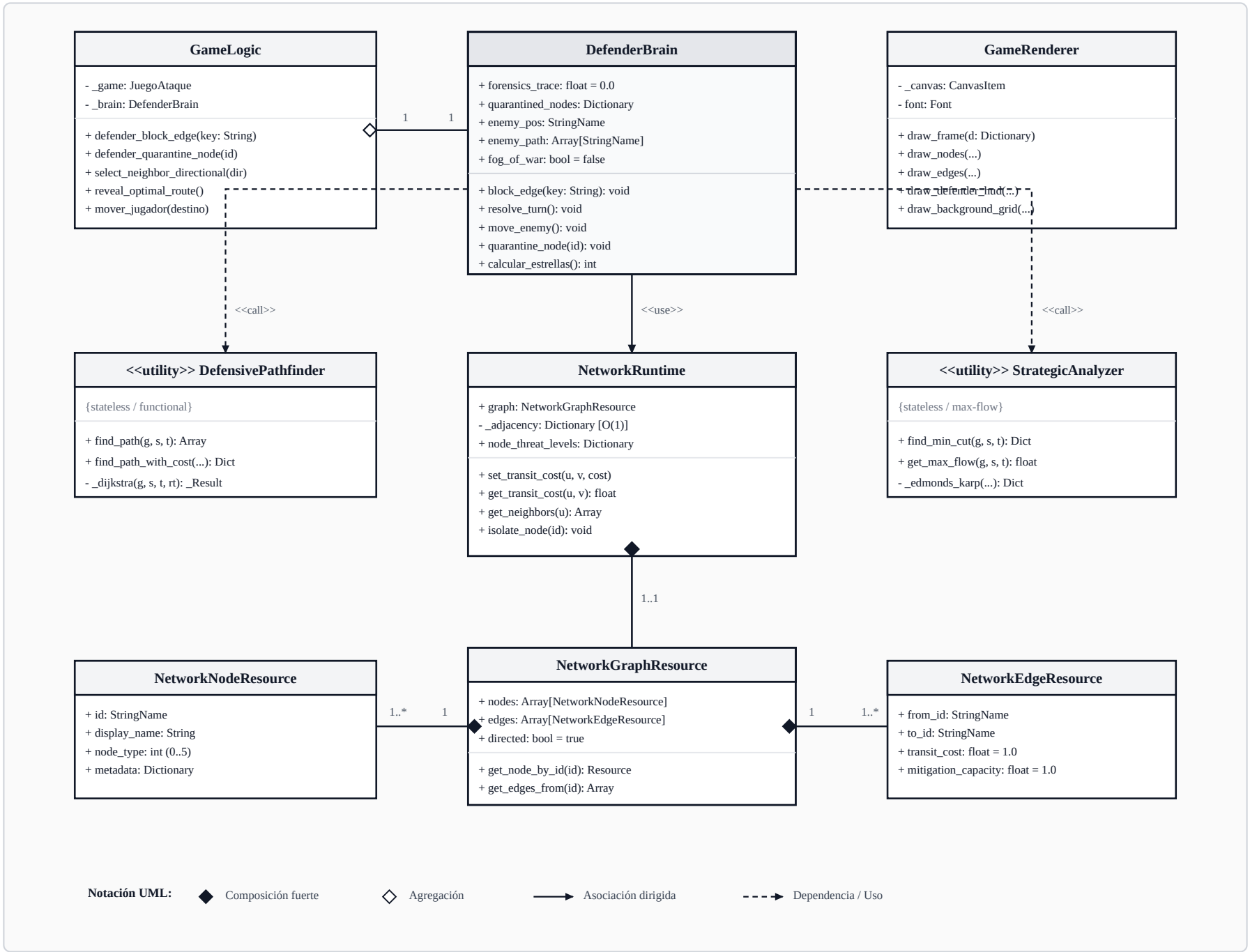

GameLogic
- _game: JuegoAtaque
- _brain: DefenderBrain
+ defender_block_edge(key: String)
+ defender_quarantine_node(id)
+ select_neighbor_directional(dir)
+ reveal_optimal_route()
+ mover_jugador(destino)
1
1
DefenderBrain
+ forensics_trace: float = 0.0
+ quarantined_nodes: Dictionary
+ enemy_pos: StringName
+ enemy_path: Array[StringName]
+ fog_of_war: bool = false
+ block_edge(key: String): void
+ resolve_turn(): void
+ move_enemy(): void
+ quarantine_node(id): void
+ calcular_estrellas(): int
GameRenderer
- _canvas: CanvasItem
- font: Font
+ draw_frame(d: Dictionary)
+ draw_nodes(...)
+ draw_edges(...)
+ draw_defender_hud(...)
+ draw_background_grid(...)
<<call>>
<<use>>
<<call>>
<<utility>> DefensivePathfinder
{stateless / functional}
+ find_path(g, s, t): Array
+ find_path_with_cost(...): Dict
- _dijkstra(g, s, t, rt): _Result
NetworkRuntime
+ graph: NetworkGraphResource
- _adjacency: Dictionary [O(1)]
+ node_threat_levels: Dictionary
+ set_transit_cost(u, v, cost)
+ get_transit_cost(u, v): float
+ get_neighbors(u): Array
+ isolate_node(id): void
<<utility>> StrategicAnalyzer
{stateless / max-flow}
+ find_min_cut(g, s, t): Dict
+ get_max_flow(g, s, t): float
- _edmonds_karp(...): Dict
1..1
NetworkNodeResource
+ id: StringName
+ display_name: String
+ node_type: int (0..5)
+ metadata: Dictionary
1..*
1
NetworkGraphResource
+ nodes: Array[NetworkNodeResource]
+ edges: Array[NetworkEdgeResource]
+ directed: bool = true
+ get_node_by_id(id): Resource
+ get_edges_from(id): Array
1
1..*
NetworkEdgeResource
+ from_id: StringName
+ to_id: StringName
+ transit_cost: float = 1.0
+ mitigation_capacity: float = 1.0
Notación UML:
Composición fuerte
Agregación
Asociación dirigida
Dependencia / Uso

Figura 5. *Diagrama de clases de arquitectura y separación estructural de responsabilidades (UML 2.5).*

Figura 6. Modelo de Datos y Persistencia de Sesión (ER / Relacional)

La **Figura 6** presenta el **diagrama de modelo de datos entidad-relación (ER / Esquema Lógico de Persistencia)** de **VERTEX**. Modela la estructura transaccional de archivos de configuración alojados en `user://`. El meta-registro `SlotMeta` gobierna el puntero de sesión activa sobre las tres ranuras independientes (`SaveSlot`). Cada ranura aísla la entidad de identidad del agente (`Profile`), el registro de maestría por escenario (`LevelProgress`), las métricas históricas de rendimiento (`GlobalStats` y `LevelStats`) y el catálogo de méritos desbloqueados (`BadgeRecord`).

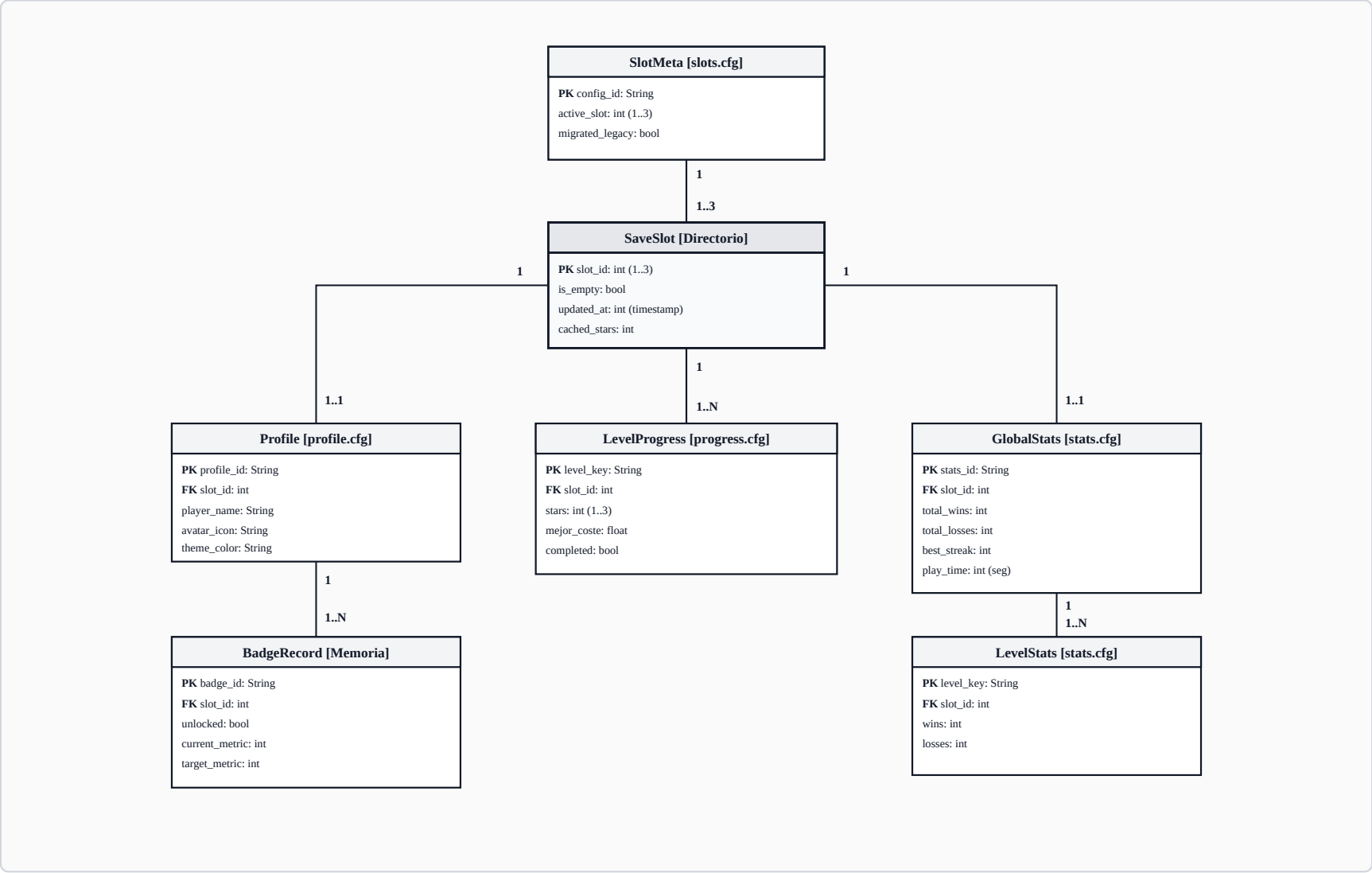


Figura 6. *Modelo relacional de entidades y esquema lógico de almacenamiento transaccional en disco.*

Figura 7. Diagrama de Máquina de Estados FSM (Dual)

La **Figura 7** ilustra el **diagrama de máquina de estados (State Machine / FSM)** dual del sistema: en la partición superior modela el ciclo de vida de los activos de red (`NetworkNode`), transitando desde el estado de operación normal hasta el bloqueo perimetral, la infección por movimiento lateral o el aislamiento físico en cuarentena; en la partición inferior modela el autómata de toma de decisiones del adversario simulado (`Adversary AI FSM`), gobernado por la detección de aristas cerradas, el consumo de exploits `Bypass` y la contención total hacia los estados terminales de brecha o mitigación forense.

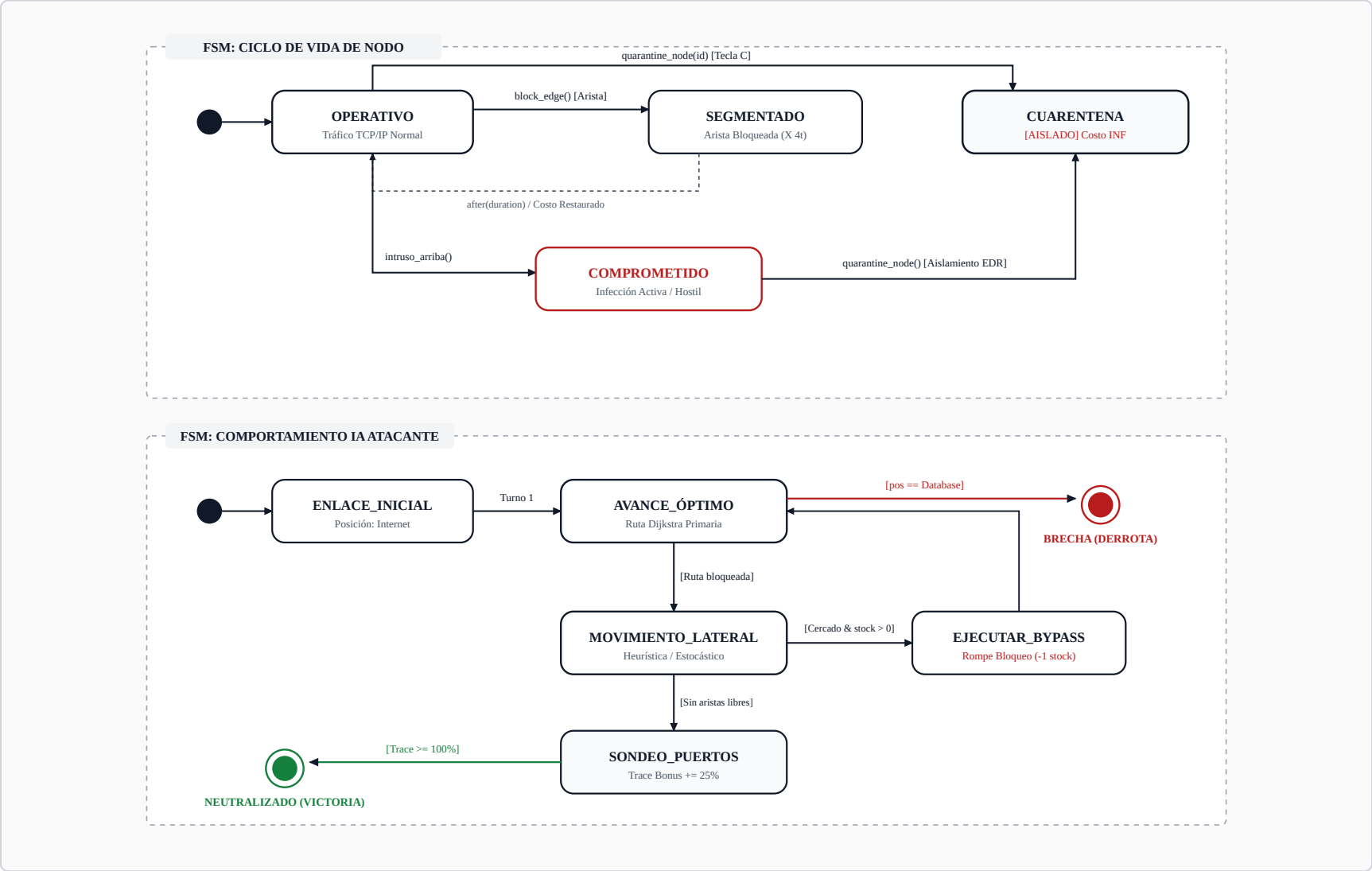


Figura 7. *Diagrama de estados (FSM) del ciclo de vida de los nodos y del autómata reactivo del adversario.*